

1.MySQL体系架构

1.1.MySQL的分支与变种

MySQL变种有好几个，主要有三个久经考验的主流变种：Percona Server，MariaDB和 Drizzle。它们都有活跃的用户社区和一些商业支持，均由独立的服务供应商支持。同时还有几个优秀的开源关系数据库，值得我们了解一下。

1.1.1.Drizzle

Drizzle是真正的MySQL分支，而且是完全开源的产品，而非只是个变种或增强版本。它并不与MySQL兼容不能简单地将MySQL后端替换为Drizzle。

Drizzle与MySQL有很大差别，进行了一些重大更改，甚至SQL语法的变化都非常大，设计目标之一是提供一种出色的解决方案来解决高可用性问题。在实现上，Drizzle清除了一些表现不佳和不必要的功能，将很多代码重写，对它们进行了优化，甚至将所用语言从C换成了C++。

此外，Drizzle另一个设计目标是能很好的适应具有大量内容的多核服务器、运行Linux的64位机器、云计算中使用的服务器、托管网站的服务器和每分钟接收数以万计点击率的服务器并且大幅度的削减服务器成本。

1.1.2.MariaDB

在Sun收购MySQL后，Monty Widenius，这位MySQL的创建者，因不认同MySQL开发流程而离开Sun。他成立了Monty程序公司，创立了MariaDB。MariaDB的目标是社区开发，Bug修复和许多的新特性实际上，可以将MariaDB视为MySQL的扩展集，它不仅提供MySQL提供的所有功能，还提供其他功能。MariaDB是原版MySQL的超集，因此已有的系统不需要任何修改就可以运行。

诸如Google，Facebook、维基百科等公司或者网站所使用了MariaDB。不过Monty公司不是以赢利为目的，而是由产品驱动的，这可能会带来问题，因为没有赢利的公司不一定能长久维持下去。

1.1.3.Percona Server

由领先的MySQL咨询公司Percona发布，Percona公司的口号就是“The Database Performance Experts”，Percona的创始人也就是《高性能MySQL》书的作者。

Percona Server是个与MySQL向后兼容的替代品，它尽可能不改变SQL语法、客户端/服务器协议和磁盘上的文件格式。任何运行在MySQL上的都可以运行在Percona Server上而不需要修改。切换到Percona Server只需要关闭MySQL和启动PerconaServer，不需要导出和重新导入数据。

Percona Server有三个主要的目标：透明，增加允许用户更紧密地查看服务器内部信息和行为的方法。比如慢查询日志中特别增加的详细信息；性能，Percona Server包含许多性能和可扩展性方面的改进，还加强了性能的可预测性和稳定性。其中主要集中于InnoDB；操作灵活性，Percona Server使操作人员和系统管理员在让MySQL作为架构的一部分而可靠并稳定运行时提供了很多便利。

一般来说，Percona Server中的许多特性会在后来的标准MySQL中出现。

国内公司阿里内部就运行了上千个Percona Server的实例。

1.2.MySQL的替代

1.2.1.Postgre SQL

PostgreSQL称自己是最先进的开源数据库，同时也是个一专多长的全栈数据库。最初是1985年在加利福尼亚大学伯克利分校开发的。

PostgreSQL 的稳定性极强，在崩溃、断电之类的灾难场景下依然可以保证数据的正确；在高并发读写，负载逼近极限下，PostgreSQL的性能指标仍可以维持双曲线甚至对数曲线，到顶峰之后不再下降，表现的非常稳定，而 MySQL 明显出现一个波峰后下滑；

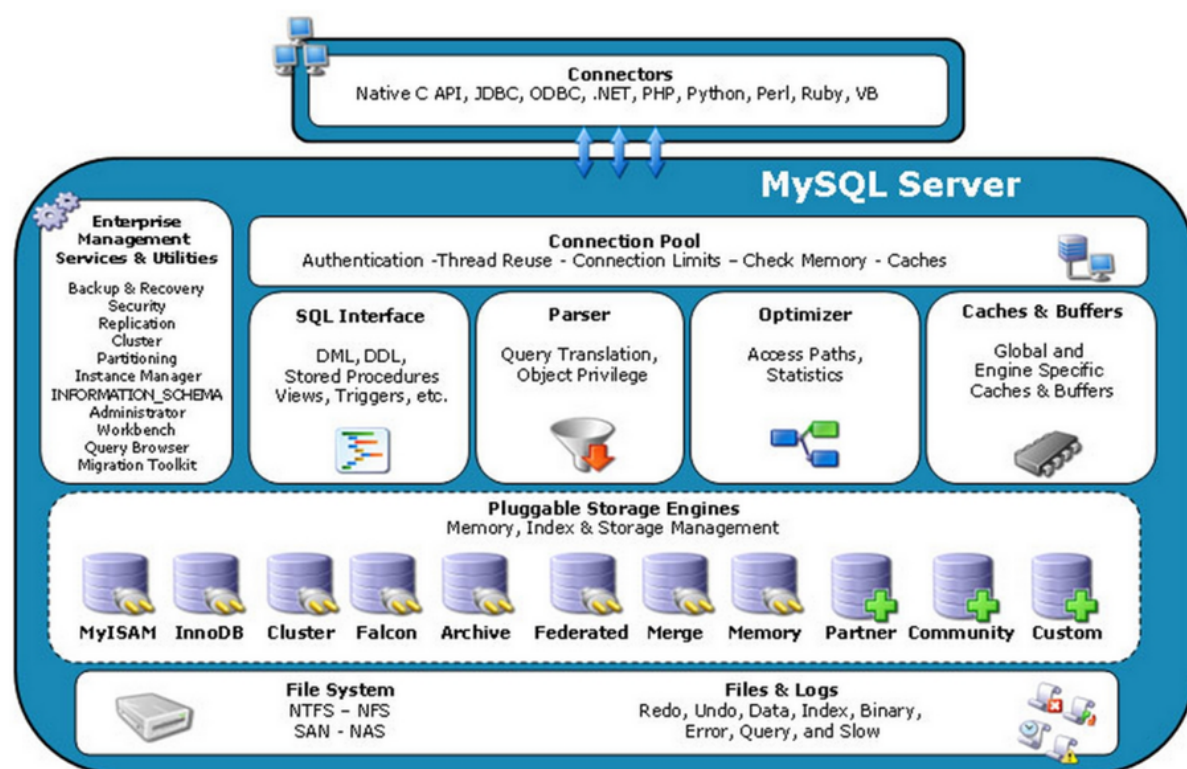
PostgreSQL多年来在GIS(地理信息)领域处于优势地位，因为它有丰富的几何类型，实际上不止几何类型，PostgreSQL有大量字典、数组、bitmap 等数据类型，相比之下mysql就差很多。所以总的来说，PostgreSQL更学术化一些，在绝对需要可靠性和数据完整性的时候，PostgreSQL是更好的选择。但是从商业支持、文档资料、易用性，第三方支持来说，MySQL无疑更好些。

1.2.2.SQLite

SQLite是世界上部署最广泛的数据库引擎，为物联网（IoT）下的数据库首选，并且是手机，PDA，甚至MP3播放器的下的首选。SQLite代码占用空间小，并且不需要数据库管理员的维护。SQLite没有单独的服务器进程，提供的事务也基本符合ACID。当然，简单也就意味着功能和性能受限。

2.MySql基础

2.1.MySQL体系架构



可以看出MySQL是由连接池、管理工具和服务、SQL接口、解析器、优化器、缓存、存储引擎、文件系统组成。

连接池

由于每次建立连接需要消耗很多时间，连接池的作用就是将这些连接缓存下来，下次可以直接用已经建立好的连接，提升服务器性能。

管理工具和服务

系统管理和控制工具，例如备份恢复、Mysql复制、集群等

SQL接口

接受用户的SQL命令，并且返回用户需要查询的结果。比如select ... from就是调用SQL接口

解析器

SQL命令传递到解析器的时候会被解析器验证和解析。解析器主要功能：1、将SQL语句分解成数据结构，后续步骤的传递和处理就是基于这个结构的。2、将SQL语句分解成数据结构，后续步骤的传递和处理就是基于这个结构的。

优化器

查询优化器，SQL语句在查询之前会使用查询优化器对查询进行优化。

缓存器

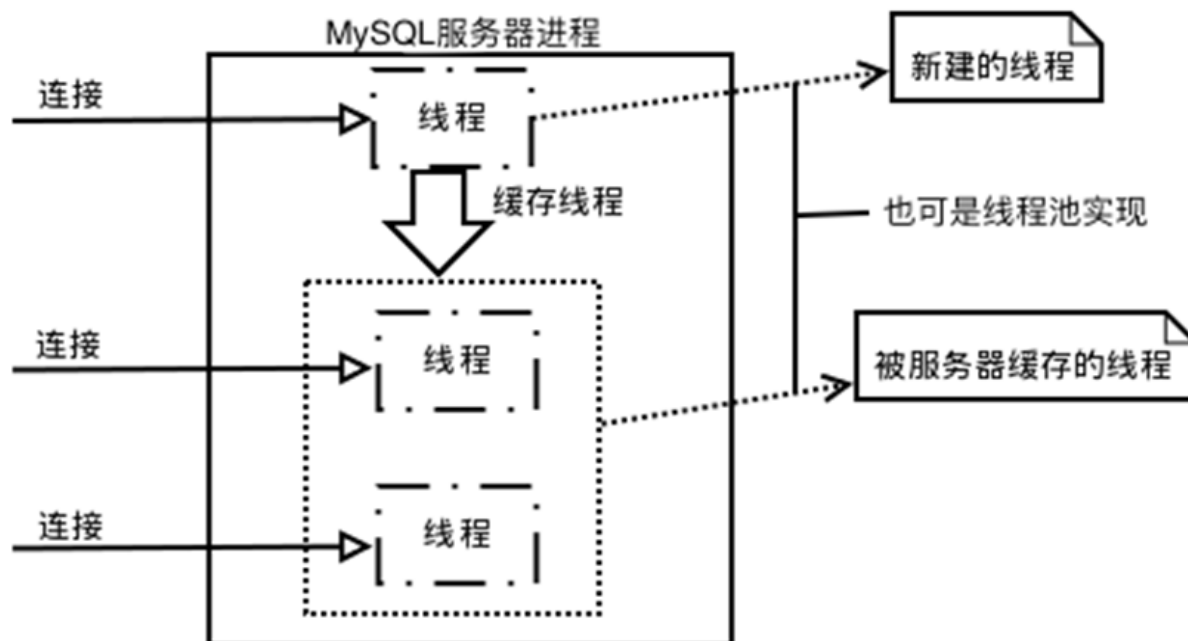
查询缓存，如果查询缓存有命中的查询结果，查询语句就可以直接去查询缓存中取数据。这个缓存机制是由一系列小缓存组成的。比如表缓存，记录缓存，key缓存，权限缓存等。

存储引擎(后面会细讲)

文件系统(后面会细讲)

2.1.1.连接层

当MySQL启动（MySQL服务器就是一个进程），等待客户端连接，每一个客户端连接请求，服务器进程会创建一个线程专门处理与这个客户端的交互。当客户端与该服务器断开之后，不会立即撤销线程，只会把他缓存起来等待下一个客户端请求连接的时候，将其分配给该客户端。每个线程独立，拥有各自的内存处理空间。

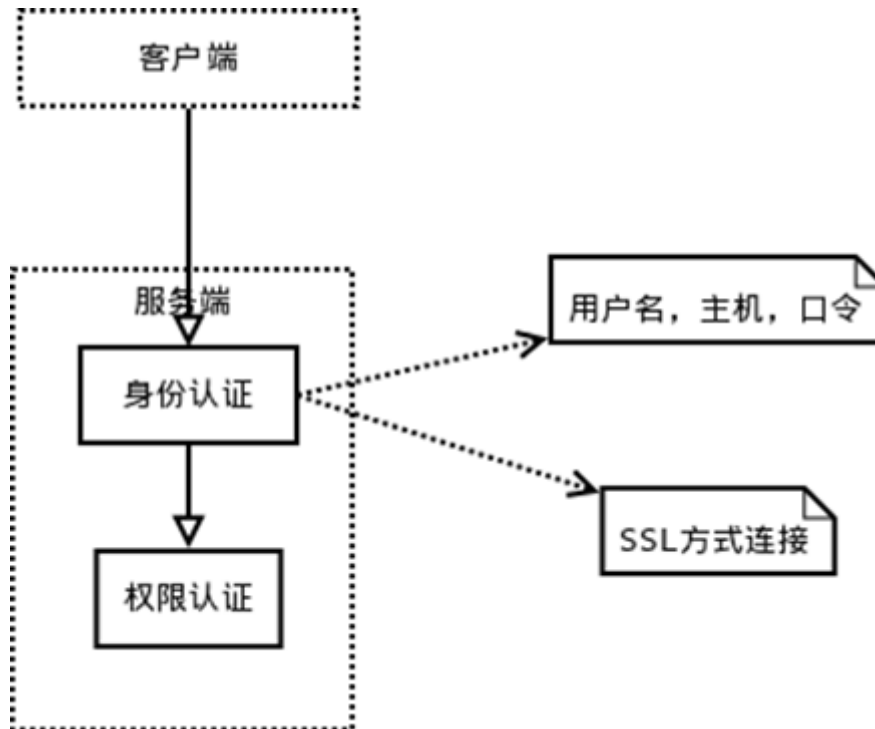


以下命令可以查看最大的连接数：

```
show VARIABLES like '%max_connections%'
```

```
mysql> show VARIABLES like '%max_connections%';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| max_connections | 151 |
+-----+-----+
1 row in set (0.00 sec)
```

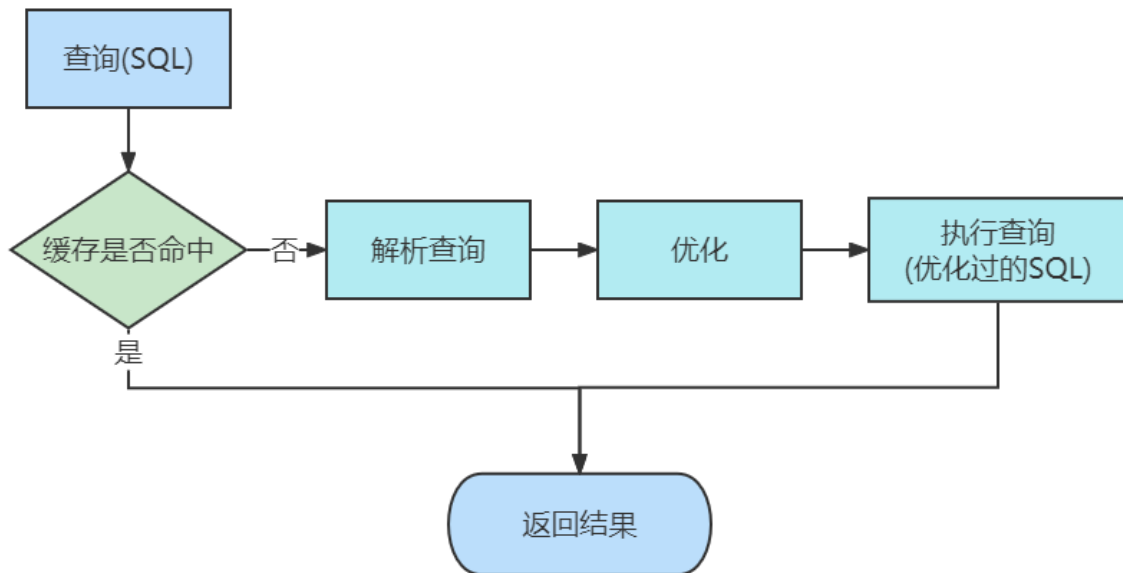
连接到服务器，服务器需要对其进行验证，也就是用户名、IP、密码验证，一旦连接成功，还要验证是否具有执行某个特定查询的权限（例如，是否允许客户端对某个数据库某个表的某个操作）



2.1.2.Server层(SQL处理层)

这一层主要功能有：SQL语句的解析、优化，缓存的查询，MySQL内置函数的实现，跨存储引擎功能（所谓跨存储引擎就是说每个引擎都需提供的功能（引擎需对外提供接口）），例如：存储过程、触发器、视图等。

当然作为一个SQL的执行流程如下：



1.如果是查询语句（select语句），首先会查询缓存是否已有相应结果，有则返回结果，无则进行下一步（如果不是查询语句，同样调到下一步）

2.解析查询，创建一个内部数据结构（解析树），这个解析树主要用来SQL语句的语义与语法解析；

3.优化：优化SQL语句，例如重写查询，决定表的读取顺序，以及选择需要的索引等。这一阶段用户是可以查询的，查询服务器优化器是如何进行优化的，便于用户重构查询和修改相关配置，达到最优化。这一阶段还涉及到存储引擎，优化器会询问存储引擎，比如某个操作的开销信息、是否对特定索引有查询优化等。

2.1.2.1.缓存（了解即可）

```
show variables like '%query_cache_type%' -- 默认不开启
```

```
show variables like '%query_cache_size%' --默认值1M
```

```
SET GLOBAL query_cache_type = 1; --会报错
```

```
mysql> show variables like '%query_cache_type%' ;
+-----+-----+
| Variable_name | Value |
+-----+-----+
| query_cache_type | OFF   |
+-----+-----+
1 row in set (0.00 sec)
```

```
mysql> show variables like '%query_cache_size%';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| query_cache_size | 1048576 |
+-----+-----+
1 row in set (0.00 sec)
```

```
mysql> SET GLOBAL query_cache_type = 1;
ERROR 1651 (HY000): Query cache is disabled; restart the server with query_cache_type=1 to enable it
mysql>
```

query_cache_type只能配置在my.cnf文件中!

缓存在生产环境建议不开启，除非经常有sql完全一模一样的查询

缓存严格要求2次SQL请求要完全一样，包括SQL语句，连接的数据库、协议版本、字符集等因素都会影响****

从8.0开始，MySQL不再使用查询缓存，那么放弃它的原因是什么呢？

MySQL查询缓存是查询结果缓存。它将以SEL开头的查询与哈希表进行比较，如果匹配，则返回上一次查询的结果。进行匹配时，查询必须逐字节匹配，例如 SELECT * FROM e1; 不等于select * from e1;

此外，一些不确定的查询结果无法被缓存，任何对表的修改都会导致这些表的所有缓存无效。因此，适用于查询缓存的最理想的方案是只读，特别是需要检查数百万行后仅返回数行的复杂查询。如果你的查询符合这样一个特点，开启查询缓存会提升你的查询性能。

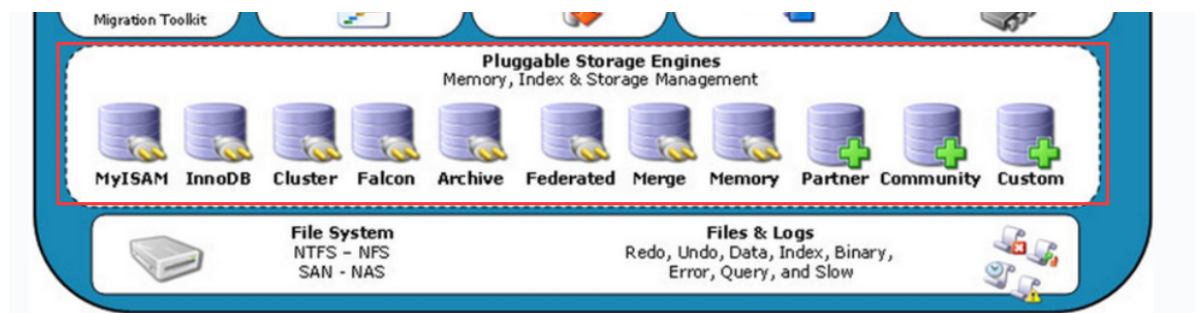
随着技术的进步，经过时间的考验，MySQL的工程团队发现启用缓存的好处并不多。

首先，查询缓存的效果取决于缓存的命中率，只有命中缓存的查询效果才能有改善，因此无法预测其性能。

其次，查询缓存的另一个大问题是它受到单个互斥锁的保护。在具有多个内核的服务器上，大量查询会导致大量的互斥锁争用。

通过基准测试发现，大多数工作负载最好禁用查询缓存(5.6的默认设置)：按照官方所说的：造成的问题比它解决问题要多的多，弊大于利就直接砍掉了。

2.1.3.存储引擎层



从体系结构图中可以发现，MySQL数据库区别于其他数据库的最重要的一个特点就是其插件式的表存储引擎。MySQL插件式的存储引擎架构提供了一系列标准的管理和服务支持，这些标准与存储引擎本身无关，可能是每个数据库系统本身都必需的，如SQL分析器和优化器等，而存储引擎是底层物理结构和实际文件读写的实现，每个存储引擎开发者可以按照自己的意愿来进行开发。**需要特别注意的是，存储引擎是基于表的，而不是数据库。**

插件式存储引擎的好处是，每个存储引擎都有各自的特点，能够根据具体的应用建立不同存储引擎表。由于MySQL数据库的开源特性，用户可以根据MySQL预定义的存储引擎接口编写自己的存储引擎。若用户对某一种存储引擎的性能或功能不满意，可以通过修改源码来得到想要的特性，这就是开源带给我们的方便与力量。

由于MySQL数据库开源特性，存储引擎可以分为MySQL官方存储引擎和第三方存储引擎。有些第三方存储引擎很强大，**如大名鼎鼎的InnoDB存储引擎（最早是第三方存储引擎，后被Oracle收购），**其应用就极其广泛，甚至是MySQL数据库OLTP(Online Transaction Processing在线事务处理)应用中使用最广泛的存储引擎。

2.1.3.1.MySQL官方引擎概要

InnoDB存储引擎

InnoDB是MySQL的默认事务型引擎，也是最重要、使用最广泛的存储引擎。它被设计用来处理大量的短期(short-lived)事务，短期事务大部分情况是正常提交的，很少会被回滚。InnoDB的性能和自动崩溃恢复特性，使得它在非事务型存储的需求中也很流行。除非有非常特别的原因需要使用其他的存储引擎，否则应该优先考虑InnoDB引擎。如果要学习存储引擎，InnoDB也是一个非常好的值得花最多的时间去深入学习的对象，收益肯定比将时间平均花在每个存储引擎的学习上要高得多。所以InnoDB引擎也将是我们学习的重点。

MyISAM存储引擎

在MySQL 5.1及之前的版本，MyISAM是默认的存储引擎。MyISAM提供了大量的特性，包括全文索引、压缩、空间函数（GIS）等，但MyISAM不支持事务和行级锁，而且有一个毫无疑问的缺陷就是崩溃后无法安全恢复。尽管MyISAM引擎不支持事务、不支持崩溃后的安全恢复，但它绝不是一无是处的。对于只读的数据，或者表比较小、可以忍受修复（repair）操作，则依然可以继续使用MyISAM（但请不要默认使用MyISAM，而是应当默认使用InnoDB）。但是MyISAM对整张表加锁，而不是针对行。读取时会对需要读到的所有表加共享锁，写入时则对表加排他锁。MyISAM很容易因为表锁的问题导致典型的性能问题。

Mrg_MyISAM

Merge存储引擎，是一组MyIsam的组合，也就是说，他将MyIsam引擎的多个表聚合起来，但是他的内部没有数据，真正的数据依然是MyIsam引擎的表中，但是可以直接进行查询、删除更新等操作。

Archive引擎

Archive存储引擎只支持INSERT和SELECT操作，在MySQL 5.1之前也不支持索引。Archive引擎会缓存所有的写并利用zlib对插入的行进行压缩，所以比MyISAM表的磁盘I/O更少。但是每次SELECT查询都需要执行全表扫描。所以Archive表适合日志和数据采集类应用，这类应用做数据分析时往往需要全表扫描。或者在一些需要更快速的INSERT操作的场合下也可以使用。Archive引擎不是一个事务型的引擎，而是一个针对高速插入和压缩做了优化的简单引擎。

Blackhole引擎

Blackhole引擎没有实现任何的存储机制，它会丢弃所有插入的数据，不做任何保存。但是服务器会记录Blackhole表的日志，所以可以用于复制数据到备库，或者只是简单地记录到日志。这种特殊的存储引擎可以在一些特殊的复制架构和日志审核时发挥作用。但这种引擎在应用方式上有很多问题，因此并不推荐。

CSV引擎

CSV引擎可以将普通的CSV文件(逗号分割值的文件)作为MySQL的表来处理，但这种表不支持索引。CSV引擎可以在数据库运行时拷入或者拷出文件。可以将Excel等的数据存储为CSV文件，然后复制到MySQL数据目录下，就能在MySQL中打开使用。同样，如果将数据写入到一个CSV引擎表，其他的外部程序也能立即从表的数据文件中读取CSV格式的数据。因此CSV引擎可以作为一种数据交换的机制，非常有用。

Federated引擎

Federated引擎是访问其他MySQL服务器的一个代理，它会创建一个到远程MySQL服务器的客户端连接，并将查询传输到远程服务器执行，然后提取或者发送需要的数据。最初设计该存储引擎是为了和企业级数据库如Microsoft SQL Server和Oracle的类似特性竞争的，可以说更多的是一种市场行为。尽管该引擎看起来提供了一种很好的跨服务器的灵活性，但也经常带来问题，因此默认是禁用的。

Memory引擎

如果需要快速地访问数据，并且这些数据不会被修改，重启以后丢失也没有关系，那么使用Memory表（以前也叫做HEAP表）是非常有用的。Memory表至少比MyISAM表要快一个数量级，因为每个基于MEMORY存储引擎的表实际对应一个磁盘文件。该文件的文件名与表名相同，类型为frm类型。该文件中只存储表的结构。而其数据文件，都是存储在内存中，这样有利于数据的快速处理，提高整个表的效率，不需要进行磁盘I/O。所以Memory表的结构在重启以后还会保留，但数据会丢失。

Memory表支持Hash索引，因此查找操作非常快。虽然Memory表的速度非常快，但还是无法取代传统的基于磁盘的表。Memory表是表级锁，因此并发写入的性能较低。它不支持BLOB或TEXT类型的列，并且每行的长度是固定的，所以即使指定了VARCHAR列，实际存储时也会转换成CHAR，这可能导致部分内存的浪费。

NDB集群引擎

使用MySQL服务器、NDB集群存储引擎，以及分布式的、share-nothing的、容灾的、高可用的NDB数据库的组合，被称为MySQL集群（MySQL Cluster）。

2.1.3.2.值得了解的第三方引擎

Percona的XtraDB存储引擎

基于InnoDB引擎的一个改进版本，已经包含在Percona Server和MariaDB中，它的改进点主要集中在性能、可测量性和操作灵活性方面。XtraDB可以作为InnoDB的一个完全的替代产品，甚至可以兼容地读写InnoDB的数据文件，并支持InnoDB的所有查询。

TokuDB引擎

使用了一种新的叫做分形树(Fractal Trees)的索引数据结构。该结构是缓存无关的，因此即使其大小超过内存性能也不会下降，也就没有内存生命周期和碎片的问题。TokuDB是一种大数据（Big Data）存储引擎，因为其拥有很高的压缩比，可以在很大的数据量上创建大量索引。现在该引擎也被Percona公司收购。

Tips：分形树，是一种写优化的磁盘索引数据结构。分形树的写操作（Insert/Update/Delete）性能比较好，同时它还能保证读操作近似于B+树的读性能。据测试结果显示，TokuDB分形树的写性能优于InnoDB的B+树，读性能略低于B+树。分形树核心思想是利用节点的MessageBuffer缓存更新操作，充分利用数据局部性原理，将随机写转换为顺序写，这样极大的提高了随机写的效率。

Infobright

MySQL默认是面向行的，每一行的数据是一起存储的，服务器的查询也是以行为单位处理的。而在大数据量处理时，面向列的方式可能效率更高，比如HBASE就是面向列存储的。

Infobright是最有名的面向列的存储引擎。在非常大的数据量（数十TB）时，该引擎工作良好。Infobright是为数据分析和数据仓库应用设计的。数据高度压缩，按照块进行排序，每个块都对应有一组元数据。在处理查询时，访问元数据可决定跳过该块，甚至可能只需要元数据即可满足查询的需求。但该引擎不支持索引，不过在这么大的数据量级，即使有索引也很难发挥作用，而且块结构也是一种准索引(quasi-index)。Infobright需要对MySQL服务器做定制，因为一些地方需要修改以适应面向列存储的需要。如果查询无法在存储层使用面向列的模式执行，则需要在服务器层转换成按行处理，这个过程会很慢。Infobright有社区版和商业版两个版本。

2.1.3.3.选择合适的引擎

这么多存储引擎，我们怎么选择？大部分情况下，InnoDB都是正确的选择，所以在MySQL 5.5版本将InnoDB作为默认的存储引擎了。对于如何选择存储引擎，可以简单地归纳为一句话：“**除非需要用到某些InnoDB不具备的特性，并且没有其他办法可以替代，否则都应该优先选择InnoDB引擎**”。比如，MySQL中只有MyISAM支持地理空间搜索。

当然，如果不需要用到InnoDB的特性，同时其他引擎的特性能够更好地满足需求，也可以考虑一下其他存储引擎。举个例子，如果不在乎可扩展能力和并发能力，也不在乎崩溃后的数据丢失问题，却对InnoDB的空间占用过多比较敏感，这种场合下选择MyISAM就比较合适。

除非万不得已，否则建议不要混合使用多种存储引擎，否则可能带来一系列复杂的问题，以及一些潜在的bug和边界问题。存储引擎层和服务器层的交互已经比较复杂，更不用说混合多个存储引擎了。至少，混合存储对一致性备份和服务器参数配置都带来了一些困难。

2.1.3.4.表引擎的转换

有很多种方法可以将表的存储引擎转换成另外一种引擎。每种方法都有其优点和缺点。常用的有三种方法

ALTER TABLE

将表从一个引擎修改为另一个引擎最简单的办法是使用ALTER TABLE 语句。下面的语句将mytable的引擎修改为InnoDB：

```
ALTER TABLE mytable ENGINE = InnoDB;
```

上述语法可以适用任何存储引擎。但需要执行很长时间，在实现上，MySQL会按行将数据从原表复制到一张新的表中，在复制期间可能会消耗系统所有的I/O能力，同时原表上会加上读锁。所以，在繁忙的表上执行此操作要特别小心。

导出与导入

还可以使用mysqldump工具将数据导出到文件，然后修改文件中CREATE TABLE语句的存储引擎选项，注意同时修改表名，因为同一个数据库中不能存在相同的表名，即使它们使用的是不同的存储引擎。

CREATE和 SELECT

先创建一个新的存储引擎的表，然后利用INSERT...SELECT语法来导出数据：

```
CREATE TABLE innodb_table LIKE myisam_table;
ALTER TABLE innodb_table ENGINE=InnoDB;
INSERT INTO innodb_table SELECT * FROM myisam_table;
```

如果数据量很大，则可以考虑做分批处理，针对每一段数据执行事务提交操作。

2.1.3.5.检查MySQL的引擎

看我的MySQL现在已提供什么存储引擎：

```
show engines;
```

```
mysql> show engines;
```

Engine	Support	Comment	Transactions	XA	Savepoints
InnoDB	DEFAULT	Supports transactions, row-level locking, and foreign keys	YES	YES	YES
MRG_MYISAM	YES	Collection of identical MyISAM tables	NO	NO	NO
MEMORY	YES	Hash based, stored in memory, useful for temporary tables	NO	NO	NO
BLACKHOLE	YES	/dev/null storage engine (anything you write to it disappears)	NO	NO	NO
MyISAM	YES	MyISAM storage engine	NO	NO	NO
CSV	YES	CSV storage engine	NO	NO	NO
ARCHIVE	YES	Archive storage engine	NO	NO	NO
PERFORMANCE_SCHEMA	YES	Performance Schema	NO	NO	NO
FEDERATED	NO	Federated MySQL storage engine	NULL	NULL	NULL

看我的MySQL当前默认的存储引擎：

```
show variables like '%storage_engine';
```

```
mysql> show variables like '%storage_engine%';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| default_storage_engine | InnoDB |
| default_tmp_storage_engine | InnoDB |
| disabled_storage_engines |  |
| internal_tmp_disk_storage_engine | InnoDB |
+-----+-----+
4 rows in set (0.00 sec)
```

2.1.3.6.MyISAM和InnoDB比较

对比项	MyISAM	InnoDB
主外键	不支持	支持
事务	不支持	支持
行表锁	表锁，即使操作一条记录也会锁住整个表 不适合高并发的操作	行锁,操作时只锁某一行，不对其它行有影响 适合高并发的操作
缓存	只缓存索引，不缓存真实数据	不仅缓存索引还要缓存真实数据，对内存要求较高，而且内存大小对性能有决定性的影响
表空间	小	大
关注点	性能	事务
默认安装	Y	Y

2.2.MySQL中的目录和文件

2.2.1.bin目录

在MySQL的安装目录下有一个特别特别重要的bin目录，这个目录下存放着许多可执行文件。

其他系统中的可执行文件与此的类似。这些可执行文件都是与服务器程序和客户端程序相关的。

```
[root@centosvm mysql-5.7.38-linux-glibc2.12-x86_64]# cd bin
[root@centosvm bin]# ls
innochecksum      mysqladmin          mysqld_multi        mysqlpump            mysqlxtest
lz4_decompress    mysqlbinlog          mysqld_safe         mysql_secure_installation  perror
myisamchk          mysqlcheck           mysqldump            mysqlshow            replace
myisam_ftdump     mysql_client_test_embedded  mysqldumpslow       mysqlslap            resolveip
myisamlog          mysql_config          mysql_embedded       mysql_ssl_rsa_setup   resolve_stack_dump
myisampack         mysql_config_editor   mysqlimport          mysqltest_embedded    zlib_decompress
my_print_defaults mysqld               mysql_install_db     mysql_tzinfo_to_sql
mysql              mysqld-debug         mysql_plugin         mysql_upgrade
```

2.2.1.1.启动MySQL服务器程序

在UNIX系统中用来启动MySQL服务器程序的可执行文件有很多，大多在MySQL安装目录的bin目录下。

mysqld

mysqld这个可执行文件就代表着MySQL服务器程序，运行这个可执行文件就可以直接启动一个服务器进程。但这个命令不常用。

mysqld_safe

mysqld safe是一个启动脚本，它会间接的调用mysqld，而且还顺便启动了另外一个监控进程，这个监控进程在服务器进程挂了的时候，可以帮助重启它。另外,使用mysqld_safe启动服务器程序时，它会将服务器程序的出错信息和其他诊断信息重定向到某个文件中，产生出错日志，这样可以方便我们找出发生错误的原因。

mysql.server

mysql.server也是一个启动脚本，它会间接的调用mysqld_safe，在调用mysql.server时在后边指定start参数就可以启动服务器程序了

就像这样:

```
mysql.server start
```

需要注意的是，这个mysql.server文件其实是一个链接文件，它的实际文件是support-files/mysql.server，所以如果在bin目录找不到，到support-files下去找找，而且如果你愿意的话，自行用ln命令在bin创建一个链接。

另外，我们还可以使用mysql.server命令来关闭正在运行的服务器程序，只要把start参数换成stop就好了:

```
mysql.server stop
```

mysqld_multi

其实我们一台计算机上也可以运行多个服务器实例，也就是运行多个MySQL服务器进程。mysqld_multi可执行文件可以对每一个服务器进程的启动或停止进行监控。

2.2.1.2.客户端程序

在我们成功启动MySQL服务器程序后，就可以接着启动客户端程序来连接到这个服务器喽，bin目录下有许多客户端程序，比方说mysqladmin、mysqldump、mysqlcheck等等。

我们常用的是可执行文件mysql，通过这个可执行文件可以让我们和服务器程序进程交互，也就是发送请求，接收服务器的处理结果。

mysqladmin执行管理操作的工具，检查服务器配置、当前运行状态，创建、删除数据库、设置新密码。

mysqldump数据库逻辑备份程序。

mysqlbackup备份数据表、整个数据库、所有数据库，一般来说mysqldump备份、mysql还原。

2.2.2.启动选项和参数

2.2.2.1.配置参数文件

当MySQL实例启动时，数据库会先去读一个配置参数文件，用来寻找数据库的各种文件所在位置以及指定某些初始化参数，这些参数通常定义了某种内存结构有多大等。在默认情况下，MySQL实例会按照-的顺序在指定的位置进行读取，用户只需通过命令mysql --help|grep my.cnf来寻找即可。

```
[root@centosvm mysql]# mysql --help|grep my.cnf
order of preference, my.cnf, $MYSQL_TCP_PORT,
/etc/my.cnf /etc/mysql/my.cnf /usr/etc/my.cnf ~/.my.cnf
```

当然，也可以在启动MySQL时，指定配置文件（非yum安装）：

```
[mysql@i2wz9j203ithc4guluwb2wZ mysql57]$ ./bin/mysqld_safe --defaults-file=/home/mysql/mysql57/conf/my.cnf --user=mysql &
[1] 18210
[mysql@i2wz9j203ithc4guluwb2wZ mysql57]$ 2021-04-05T02:52:39.268767Z mysqld_safe Logging to '/home/mysql/mysql57/log/error.log'.
2021-04-05T02:52:39.302753Z mysqld_safe Starting mysqld daemon with databases from /home/mysql/mysql57/data
```

这个时候，就会以启动时指定的配置文件为准。

```
[mysql@i2wz9j203ithc4guluwb2wZ mysql57]$ ps -ef|grep mysqld
mysql      18210 18090  0 10:52 pts/1    00:00:00 /bin/sh ./bin/mysqld_safe --defaults-file=/home/mysql/mysql57/conf/my.cnf --user=mysql
mysql      18379 18210  0 10:52 pts/1    00:00:00 /home/mysql/mysql57/bin/mysqld --defaults-file=/home/mysql/mysql57/conf/my.cnf --basedir=/home/mysql/mysql57 --datadir=/home/mysql/mysql57/data --plugin-dir=/home/mysql/mysql57/lib/plugin --log-error=/home/mysql/mysql57/log/error.log --pid-file=/home/mysql/mysql57/log/mysql.pid --socket=/home/mysql/mysql57/log/mysql.sock --port=3306
```

MySQL数据库参数文件的作用和Oracle数据库的参数文件极其类似，不同的是，Oracle实例在启动时若找不到参数文件，是不能进行装载(mount)操作的。MySQL稍微有所不同，MySQL实例可以不需要参数文件，这时所有的参数值取决于编译MySQL时指定的默认值和源代码中指定参数的默认值。

MySQL数据库的参数文件是以文本方式进行存储的。可以直接通过一些常用的文本编辑软件进行参数的修改。

2.2.2.2.参数的查看和修改

可以通过命令show variables查看数据库中的所有参数，也可以通过LIKE来过滤参数名，前面查找数据库引擎时已经展示过了。从MySQL 5.1版本开始，还可以通过information_schema架构下的GLOBAL_VARIABLES视图来进行查找，推荐使用命令show variables，使用更为简单，且各版本的MySQL数据库都支持。

```
mysql> show variables like 'log_bin';
ERROR 2006 (HY000): MySQL server has gone away
No connection. Trying to reconnect...
Connection id:      2
Current database: mysql

+-----+-----+
| Variable_name | Value |
+-----+-----+
| log_bin       | ON    |
+-----+-----+
1 row in set (0.01 sec)
```

参数的具体含义可以参考MySQL官方手册：

<https://dev.mysql.com/doc/refman/5.7/en/server-system-variables.html>

但是课程中遇到的参数会进行讲解。

MySQL数据库中的参数可以分为两类:动态(dynamic) 参数和静态(static) 参数。同时从作用范围又可以分为全局变量和会话变量。

动态参数意味着可以在MySQL实例运行中进行更改，静态参数说明在整个实例生命周期内都不得进行更改，就好像是只读(read only)的。

全局变量 (GLOBAL) 影响服务器的整体操作。

会话变量 (SESSION/LOCAL) 影响某个客户端连接的操作。

举个例子，用default_storage_engine来说明，在服务器启动时会初始化一个名为default_storage_engine，作用范围为GLOBAL的系统变量。之后每当有一个客户端连接到该服务器时，服务器都会单独为该客户端分配一个名为default_storage_engine，作用范围为SESSION的系统变量，该作用范围为SESSION的系统变量值按照当前作用范围为GLOBAL的同名系统变量值进行初始化。

可以通过SET命令对动态的参数值进行修改。

```
mysql> set global general_log = on;
Query OK, 0 rows affected (0.00 sec)
```

SET的语法如下：

```
set [global || session ] system_var_name= expr
或者
set @@global. || @@session. system_var_name= expr
比如：
set read_buffer_size=524288;
set session read_buffer_size=524288;
set @@global.read_buffer_size=524288;
```

MySQL所有动态变量的可修改范围，可以参考MySQL官方手册的 Dynamic System Variables 的相关内容：

<https://dev.mysql.com/doc/refman/5.7/en/dynamic-system-variables.html>

对于静态变量，若对其进行修改，会得到类似如下错误：

```
mysql> set global datadir = '/db/mysql';
ERROR 1238 (HY000): Variable 'datadir' is a read only variable
```

2.2.3.数据目录

我们知道像InnoDB、MyISAM这样的存储引擎都是把表存储在磁盘上的，而操作系统用来管理磁盘的那个东东又被称为文件系统，所以用专业一点的话来表述就是：像InnoDB、MyISAM这样的存储引擎都是把表存储在文件系统上的。当我们想读取数据的时候，这些存储引擎会从文件系统中把数据读出来返回给我们，当我们想写入数据的时候，这些存储引擎会把这些数据又写回文件系统。

2.2.3.1.确定MySQL中的数据目录

那说了半天，到底MySQL把数据都存到哪个路径下呢？其实数据目录对应着一个系统变量datadir，我们在使用客户端与服务器建立连接之后查看这个系统变量的值就可以了：

```
show variables like 'datadir';
```

```
mysql> show variables like 'datadir';
+-----+-----+
| Variable_name | Value                |
+-----+-----+
| datadir       | /var/lib/mysql/     |
+-----+-----+
1 row in set (0.00 sec)
```

当然这个目录可以通过配置文件进行修改，由我们自己进行指定。

2.2.3.2.数据目录中放些什么？

MySQL在运行过程中都会产生哪些数据呢？当然会包含我们创建的数据库、表、视图和触发器等用户数据，除了这些用户数据，为了程序更好的运行，MySQL也会创建一些其他的额外数据

2.2.3.2.1.数据库在文件系统中的表示

```
create database lijn charset=utf8;
```

```
mysql> create database lijn charset=utf8;  
Query OK, 1 row affected (0.00 sec)
```

每当我们使用CREATE DATABASE语句创建一个数据库的时候，在文件系统上实际发生了什么呢？其实很简单，每个数据库都对应数据目录下的一个子目录，或者说对应一个文件夹，我们每当我们新建一个数据库时，MySQL会帮我们做这两件事儿：

1. 在数据目录下创建一个和数据库名同名的子目录（或者说是文件夹）。
2. 在该与数据库名同名的子目录下创建一个名为db.opt的文件，这个文件中包含了该数据库的各种属性，比方说该数据库的字符集和比较规则是个啥。

```
[root@centosvm mysql]# cd lijn  
[root@centosvm lijn]# ls  
db.opt
```

比方说我们查看一下在我的计算机上当前有哪些数据库：

```
mysql> show databases;  
+-----+  
| Database |  
+-----+  
| information_schema |  
| lijn |  
| mysql |  
| performance_schema |  
| sys |  
+-----+  
5 rows in set (0.00 sec)
```

可以看到在当前有5个数据库，其中mysqladv数据库是我们自定义的，其余4个数据库是属于MySQL自带的系统数据库。我们再看一下数据目录下的内容：


```

[root@centosvm mysql]# ls -l
总用量 122972
-rw-r-----. 1 mysql mysql      56 5月 13 18:17 auto.cnf
-rw-----. 1 mysql mysql     1676 5月 13 18:17 ca-key.pem
-rw-r--r--. 1 mysql mysql     1112 5月 13 18:17 ca.pem
-rw-r-----. 1 mysql mysql    18851 5月 13 22:47 centosvm-slow.log
-rw-r--r--. 1 mysql mysql     1112 5月 13 18:17 client-cert.pem
-rw-----. 1 mysql mysql     1676 5月 13 18:17 client-key.pem
-rw-r-----. 1 mysql mysql      393 5月 31 16:08 ib_buffer_pool
-rw-r-----. 1 mysql mysql 12582912 6月 10 16:44 ibdata1
-rw-r-----. 1 mysql mysql 50331648 6月 10 16:44 ib_logfile0
-rw-r-----. 1 mysql mysql 50331648 5月 13 18:17 ib_logfile1
-rw-r-----. 1 mysql mysql 12582912 6月 10 16:44 ibtmp1
drwxr-x---. 2 mysql mysql      20 6月 10 17:26 lijin
drwxr-x---. 2 mysql mysql    4096 5月 27 20:56 mysql
srwxrwxrwx. 1 mysql mysql       0 6月 10 16:44 mysql.sock
-rw-----. 1 mysql mysql       5 6月 10 16:44 mysql.sock.lock
drwxr-x---. 2 mysql mysql    8192 5月 13 18:17 performance_schema
-rw-----. 1 mysql mysql     1676 5月 13 18:17 private_key.pem
-rw-r--r--. 1 mysql mysql      452 5月 13 18:17 public_key.pem
-rw-r--r--. 1 mysql mysql     1112 5月 13 18:17 server-cert.pem
-rw-----. 1 mysql mysql     1680 5月 13 18:17 server-key.pem
drwxr-x---. 2 mysql mysql    8192 5月 13 18:17 sys
[root@centosvm mysql]# cd lijin
[root@centosvm mysql]# cd lijin
[root@centosvm lijin]# ls
db.opt

```

当然这个数据目录下的文件和子目录比较多，但是如果仔细看的话，除了information_schema这个系统数据库外，其他的数据库在数据目录下都有对应的子目录。这个information_schema比较特殊，我们后面再讲它的作用。

2.2.3.2.2.表在文件系统上的表示

我们的数据其实都是以记录的形式插入到表中的，每个表的信息其实可以分为两种：

1.表结构的定义

2. 表中的数据

表结构就是该表的名称是啥，表里边有多少列，每个列的数据类型是啥，有啥约束条件和索引，用的是啥字符集和比较规则各种信息，这些信息都体现在了我们的建表语句中了。为了保存这些信息，InnoDB和MyISAM这两种存储引擎都在数据目录下对应的数据库子目录下创建了一个专门用于描述表结构的文件，文件名是这样:表名.frm

比方说我们在lijin数据库下创建一个名为test的表：

```

mysql> use lijin
Database changed
mysql> create table test (c1 int);
Query OK, 0 rows affected (0.02 sec)

```

那在数据库mysqladv对应的子目录下就会创建一个名为test.frm的用于描述表结构的文件。这个后缀名为.frm是以二进制格式存储的。


```
[root@centosvm lijn]# ls
db.opt test.frm test.ibd
[root@centosvm lijn]#
```

那表中的数据存到什么文件中了呢?在这个问题上,不同的存储引擎就产生了有所不同,下边我们分别看一下InnoDB和MyISAM是用什么文件来保存表中数据的。

2.2.3.2.3.InnoDB是如何存储表数据的

InnoDB的数据会放在一个表空间或者文件空间(英文名: table space或者file space)的概念,这个表空间是一个抽象的概念,它可以对应文件系统上一个或多个真实文件(不同表空间对应的文件数量可能不同)。每一个表空间可以被划分为很多很多很多个页,我们的表数据就存放在某个表空间下的某些页里。表空间有好几种类型。

系统表空间(system tablespace)

这个所谓的系统表空间可以对应文件系统上一个或多个实际的文件,默认情况下,InnoDB会在数据目录下创建一个名为ibdata1(在你的数据目录下找找看有木有)、大小为12M的文件,这个文件就是对应的系统表空间在文件系统上的表示。

```
总用量 122972
-rw-r-----. 1 mysql mysql      56 5月 13 18:17 auto.cnf
-rw-r-----. 1 mysql mysql     1676 5月 13 18:17 ca-key.pem
-rw-r--r--. 1 mysql mysql     1112 5月 13 18:17 ca.pem
-rw-r-----. 1 mysql mysql    18851 5月 13 22:47 centosvm-slow.log
-rw-r--r--. 1 mysql mysql     1112 5月 13 18:17 client-cert.pem
-rw-r-----. 1 mysql mysql     1676 5月 13 18:17 client-key.pem
-rw-r-----. 1 mysql mysql      393 5月 31 16:08 ib_buffer_pool
-rw-r-----. 1 mysql mysql    12582912 6月 10 18:07 ibdata1
-rw-r-----. 1 mysql mysql    50331648 6月 10 18:07 ib_logfile0
-rw-r-----. 1 mysql mysql    50331648 5月 13 18:17 ib_logfile1
-rw-r-----. 1 mysql mysql   12582912 6月 10 16:44 ibtmp1
drwxr-x---. 2 mysql mysql      52 6月 10 18:07 lijn
drwxr-x---. 2 mysql mysql    4096 5月 27 20:56 mysql
srwxrwxrwx. 1 mysql mysql      0 6月 10 16:44 mysql.sock
-rw-r-----. 1 mysql mysql      5 6月 10 16:44 mysql.sock.lock
drwxr-x---. 2 mysql mysql    8192 5月 13 18:17 performance_schema
```

这个文件是所谓的自扩展文件,也就是当不够用的时候它会自己增加文件大小,当然,如果你想让系统表空间对应文件系统上多个实际文件,或者仅仅觉得原来的ibdata1这个文件名难听,那可以在MySQL启动时配置对应的文件路径以及它们的大小,我们也可以把系统表空间对应的文件路径不配置到数据目录下,甚至可以配置到单独的磁盘分区上。

需要注意的一点是,在一个MySQL服务器中,系统表空间只有一份。从MySQL5.5.7到MySQL5.6.6之间的各个版本中,我们表中的数据都会被默认存储到这个系统表空间。

独立表空间(file-per-table tablespace)

在MySQL5.6.6以及之后的版本中,InnoDB并不会默认的把各个表的数据存储到系统表空间中,而是为每一个表建立一个独立表空间,也就是说我们创建了多少个表,就有多少个独立表空间。使用独立表空间来存储表数据的话,会在该表所属数据库对应的子目录下创建一个表示该独立表空间的文件,文件名和表名相同,只不过添加了一个.ibd的扩展名而已,所以完整的文件名称长这样:表名.ibd。

比方说假如我们使用了独立表空间去存储lijn数据库下的test表的话,那么在该表所在数据库对应的lijn目录下会为test表创建这两个文件:

test.frm和test.ibd

```
[root@centosvm lijin]# ls -l
总用量 112
-rw-r-----. 1 mysql mysql    61 6月  10 17:26 db.opt
-rw-r-----. 1 mysql mysql  8556 6月  10 18:07 test.frm
-rw-r-----. 1 mysql mysql 98304 6月  10 18:07 test.ibd
```

其中test.ibd文件就用来存储test表中的数据和索引。当然我们也可以自己指定使用系统表空间还是独立表空间来存储数据，这个功能由启动参数

innodb_file_per_table控制，比如说我们想刻意将表数据都存储到系统表空间时，可以在启动MySQL服务器的时候这样配置：

```
[server]

innodb_file_per_table=0
```

当innodb_file_per_table的值为0时，代表使用系统表空间；当innodb_file_per_table的值为1时，代表使用独立表空间。不过innodb_file_per_table参数只对新建的表起作用，对于已经分配了表空间的表并不起作用。

其他类型的表空间

随着MySQL的发展，除了上述两种老牌表空间之外，现在还新提出了一些不同类型的表空间，比如通用表空间(general tablespace)、undo表空间(undotablespace)、临时表空间(temporary tablespace)等。

2.2.3.2.4.MyISAM是如何存储表数据的

在MyISAM中的数据和索引是分开存放的。所以在文件系统中也是使用不同的文件来存储数据文件和索引文件。而且和InnoDB不同的是，MyISAM并没有什么所谓的表空间一说，表数据都存放对应的数据库子目录下。

```
mysql> create table test_myisam(c1 int) engine =MyISAM;
Query OK, 0 rows affected (0.00 sec)
```

test_myisam表使用MyISAM存储引擎的话，那么在它所在数据库对应的lijin目录下会为myisam表创建三个文件：

```
[root@centosvm lijin]# ls -l
总用量 128
-rw-r-----. 1 mysql mysql    61 6月  10 17:26 db.opt
-rw-r-----. 1 mysql mysql  8556 6月  10 18:07 test.frm
-rw-r-----. 1 mysql mysql 98304 6月  10 18:07 test.ibd
-rw-r-----. 1 mysql mysql  8556 6月  10 18:14 test_myisam.frm
-rw-r-----. 1 mysql mysql    0 6月  10 18:14 test_myisam.MYD
-rw-r-----. 1 mysql mysql  1024 6月  10 18:14 test_myisam.MYI
```

其中test_myisam.MYD代表表的数据文件，也就是我们插入的用户记录；test_myisam.MYI代表表的索引文件，我们为表创建的索引都会放到这个文件中。

2.2.3.3.日志文件

在服务器运行过程中，会产生各种各样的日志，比如常规的查询日志、错误日志、二进制日志、redo日志、Undo日志等等，日志文件记录了影响MySQL数据库的各种类型活动。

常见的日志文件有：错误日志（error log）、慢查询日志（slow query log）、查询日志（query log）、二进制文件（bin log）。

错误日志

错误日志文件对MySQL的启动、运行、关闭过程进行了记录。遇到问题时应该首先查看该文件以便定位问题。该文件不仅记录了所有的错误信息，也记录一些警告信息或正确的信息

用户可以通过下面命令来查看错误日志文件的位置：

```
show variables like 'log_error'\G;
```

```
mysql> show variables like 'log_error'\G;
***** 1. row *****
Variable_name: log_error
Value: /var/log/mysqld.log
1 row in set (0.00 sec)
```

当MySQL不能正常启动时，第一个必须查找的文件应该就是错误日志文件，该文件记录了错误信息。

慢查询日志

慢查询日志可以帮助定位可能存在问题的SQL语句，从而进行SQL语句层面的优化。

我们已经知道慢查询日志可以帮助定位可能存在问题的SQL语句，从而进行SQL语句层面的优化。但是默认值为关闭的，需要我们手动开启。

```
show VARIABLES like 'slow_query_log';
```

1 show VARIABLES like 'slow_query_log';

2

信息

结果1

概况

状态

Variable_name	Value
slow_query_log	ON

```
set GLOBAL slow_query_log=1;
```

开启1，关闭0

但是多慢算慢？MySQL中可以设定一个阈值，将运行时间超过该值的所有SQL语句都记录到慢查询日志中。long_query_time参数就是这个阈值。默认值为10，代表10秒。

```
show VARIABLES like '%long_query_time%';
```

当然也可以设置

```
set global long_query_time=0;
```

默认10秒，这里为了演示方便设置为0

同时对于运行的SQL语句没有使用索引，则MySQL数据库也可以将这条SQL语句记录到慢查询日志文件，控制参数是：

```
show VARIABLES like '%log_queries_not_using_indexes%';
```

信息	结果1	概况	状态
Variable_name		Value	
log_queries_not_using_indexes		OFF	

开启1，关闭0（默认）

```
1 show VARIABLES like '%slow_query_log_file%';
```

信息	结果1	概况	状态
Variable_name		Value	
slow_query_log_file		DESKTOP-E5G6AHJ-slow.log	

```
show VARIABLES like '%slow_query_log_file%';
```

```
mysql> show VARIABLES like '%slow_query_log_file%';
+-----+-----+
| Variable_name | Value                                |
+-----+-----+
| slow_query_log_file | /var/lib/mysql/centosvm-slow.log |
+-----+-----+
1 row in set (0.00 sec)
```

```
[root@centosvm lijn]# cat /var/log/mysqld.log
2022-05-13T10:17:45.305396Z 0 [Warning] TIMESTAMP with implicit DEFAULT value is deprecated. Use --explicit_defaults_for_timestamp server option (see documentation for more details).
2022-05-13T10:17:45.629863Z 0 [Warning] InnoDB: New log files created, LSN=45790
2022-05-13T10:17:45.724682Z 0 [Warning] InnoDB: Creating foreign key constraint system tables
2022-05-13T10:17:45.799683Z 0 [Warning] No existing UUID has been found, so we assume that this is the first time that this server has been started. Generating a new UUID: eeb62e1a-d2a0-9800273c8838.
```

查询日志

查看当前的通用日志文件是否开启

```
show variables like '%general%'
```

```
mysql> show variables like '%general%'
-> ;
```

Variable_name	Value
general_log	OFF
general_log_file	/var/lib/mysql/centosvm.log

```
2 rows in set (0.00 sec)
```

开启通用日志查询: `set global general_log = on;`
 关闭通用日志查询: `set global general_log = off;`

```
mysql> set global general_log = on;
Query OK, 0 rows affected (0.00 sec)

mysql> show variables like '%general%';
```

Variable_name	Value
general_log	ON
general_log_file	/var/lib/mysql/centosvm.log

```
2 rows in set (0.00 sec)
```

sele

查询日志记录了所有对MySQL数据库请求的信息, 无论这些请求是否得到了正确的执行。

默认文件名: 主机名.log

```
总用量 122976
```

-rw-r-----.	1	mysql	mysql	56	5月	13	18:17	auto.cnf
-rw-----.	1	mysql	mysql	1676	5月	13	18:17	ca-key.pem
-rw-r--r--.	1	mysql	mysql	1112	5月	13	18:17	ca.pem
-rw-r-----.	1	mysql	mysql	360	6月	10	18:26	centosvm.log
-rw-r-----.	1	mysql	mysql	18851	5月	13	22:47	centosvm-slow.log
-rw-r--r--.	1	mysql	mysql	1112	5月	13	18:17	client-cert.pem
-rw-----.	1	mysql	mysql	1676	5月	13	18:17	client-key.pem
-rw-r-----.	1	mysql	mysql	393	5月	31	16:08	ib_buffer_pool
-rw-r-----.	1	mysql	mysql	12582912	6月	10	18:07	ibdata1
-rw-r-----.	1	mysql	mysql	50331648	6月	10	18:07	ib_logfile0
-rw-r-----.	1	mysql	mysql	50331648	5月	13	18:17	ib_logfile1
-rw-r-----.	1	mysql	mysql	12582912	6月	10	16:44	ibtmp1
drwxr-x---	2	mysql	mysql	121	6月	10	18:14	lijin
drwxr-x---	2	mysql	mysql	4096	5月	27	20:56	mysql
srwxrwxrwx.	1	mysql	mysql	0	6月	10	16:44	mysql.sock
-rw-----.	1	mysql	mysql	5	6月	10	16:44	mysql.sock.lock
drwxr-x---	2	mysql	mysql	8192	5月	13	18:17	performance_schema
-rw-----.	1	mysql	mysql	1676	5月	13	18:17	private_key.pem
-rw-r--r--.	1	mysql	mysql	452	5月	13	18:17	public_key.pem
-rw-r--r--.	1	mysql	mysql	1112	5月	13	18:17	server-cert.pem
-rw-----.	1	mysql	mysql	1680	5月	13	18:17	server-key.pem
drwxr-x---	2	mysql	mysql	8192	5月	13	18:17	sys

二进制日志 (binlog)

二进制日志记录了所有的DDL和DML语句（除了数据查询语句select），以事件形式记录，还包含语句所执行的消耗的时间，MySQL的二进制日志是事务安全型的

二进制日志的几种作用：

恢复（recovery）：某些数据的恢复需要二进制日志，例如，在一个数据库全备文件恢复后，用户可以通过二进制文件进行point-in-time的恢复

复制（replication）：其原理与恢复类似，通过复制和执行二进制日志使一台远程的MySQL数据库（一般称为slave或standby）与一台MySQL数据库（一般称为master或primary）进行实时同步

审计（audit）：用户可以通过二进制日志中的信息来进行审计，判断是否有对数据库进行注入的攻击

log-bin参数该参数用来控制是否开启二进制日志，默认为关闭

如果想要开启二进制日志的功能，可以在MySQL的配置文件中指定如下的格式：

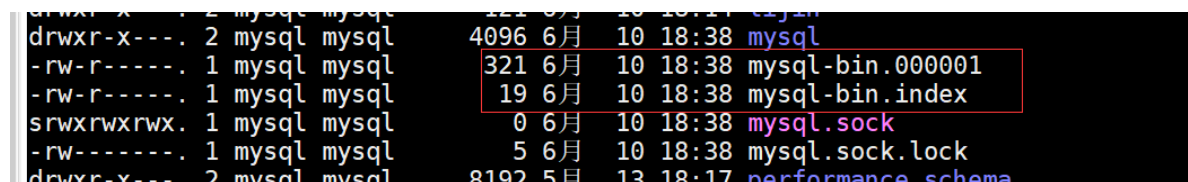
“name”为二进制日志文件的名称

如果不提供name，那么数据库会使用默认的日志文件名（文件名为主机名，后缀名为二进制日志的序列号），且文件保存在数据库所在的目录（datadir下）

--启用/设置二进制日志文件(name可省略)

log-bin=name;

配置以后，就会在数据目录下产生类似于：



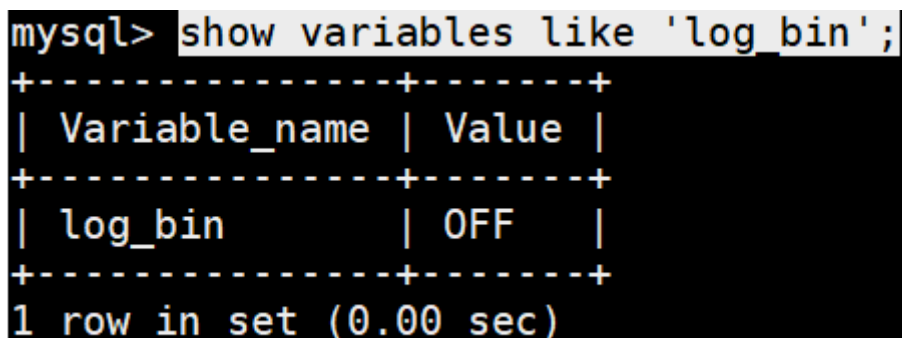
drwxr-x---	2	mysql	mysql	4096	6月	10	18:38	mysql
-rw-r-----	1	mysql	mysql	321	6月	10	18:38	mysql-bin.000001
-rw-r-----	1	mysql	mysql	19	6月	10	18:38	mysql-bin.index
srwxrwxrwx	1	mysql	mysql	0	6月	10	18:38	mysql.sock
-rw-----	1	mysql	mysql	5	6月	10	18:38	mysql.sock.lock
drwxr-x---	2	mysql	mysql	8192	5月	13	18:17	performance_schema

bin_log.000001即为二进制日志文件；bin_log.index为二进制的索引文件，用来存储过往产生的二进制日志序号，通常情况下，不建议手动修改这个文件。

二进制日志文件在默认情况下并没有启动，需要手动指定参数来启动。开启这个选项会对MySQL的性能造成影响，但是性能损失十分有限。根据MySQL官方手册中的测试指明，开启二进制日志会使性能下降1%。

查看binlog是否开启

```
show variables like 'log_bin';
```



```
mysql> show variables like 'log_bin';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| log_bin      | OFF   |
+-----+-----+
1 row in set (0.00 sec)
```

mysql安装目录下修改my.cnf


```
log_bin=mysql-bin
binlog-format=ROW
server-id=1
expire_logs_days =30
```

```
# Disabling symbolic-links is recommended to prevent a
symbolic-links=0
```

```
log-error=/var/log/mysqld.log
pid-file=/var/run/mysqld/mysqld.pid
log_bin=mysql-bin
binlog-format=ROW
server-id=1
expire_logs_days=30
```

```
drwxr-x---. 2 mysql mysql 4096 6月 10 18:38 mysql
-rw-r-----. 1 mysql mysql 321 6月 10 18:38 mysql-bin.000001
-rw-r-----. 1 mysql mysql 19 6月 10 18:38 mysql-bin.index
srwxrwxrwx. 1 mysql mysql 0 6月 10 18:38 mysql.sock
-rw-----. 1 mysql mysql 5 6月 10 18:38 mysql.sock.lock
drwxr-x---. 2 mysql mysql 8192 5月 13 18:17 performance_schema
```

2.2.3.3.其他的数据文件

除了我们上边说的这些用户自己存储的数据以外，数据文件下还包括为了更好运行程序的一些额外文件，当然这些文件不一定会放在数据目录下，而且可以在配置文件或者启动时另外指定存放目录。

主要包括这几种类型的文件：

- 服务器进程文件。

我们知道每运行一个MySQL服务器程序，都意味着启动一个进程。MySQL服务器会把自己的进程ID写入到一个pid文件中。

socket文件

当用UNIX域套接字方式进行连接时需要的文件。

```
rw-r-----. 1 mysql mysql 50331648 6月 10 18:39 ib_logfile0
rw-r-----. 1 mysql mysql 50331648 5月 13 18:17 ib_logfile1
rw-r-----. 1 mysql mysql 12582912 6月 10 18:38 ibtmp1
rwxr-x---. 2 mysql mysql 121 6月 10 18:14 lijin
rwxr-x---. 2 mysql mysql 4096 6月 10 18:38 mysql
-rw-r-----. 1 mysql mysql 321 6月 10 18:38 mysql-bin.000001
-rw-r-----. 1 mysql mysql 19 6月 10 18:38 mysql-bin.index
rwxrwxrwx. 1 mysql mysql 0 6月 10 18:38 mysql.sock
-rw-----. 1 mysql mysql 5 6月 10 18:38 mysql.sock.lock
rwxr-x---. 2 mysql mysql 8192 5月 13 18:17 performance_schema
-rw-----. 1 mysql mysql 1676 5月 13 18:17 private_key.pem
-rw-r--r--. 1 mysql mysql 452 5月 13 18:17 public_key.pem
-rw-r--r--. 1 mysql mysql 1112 5月 13 18:17 server-cert.pem
```

- 默认/自动生成的SSL和RSA证书和密钥文件。

```
-rw-r-----. 1 mysql mysql 50331648 5月 13 18:17 ib_logfile1
-rw-r-----. 1 mysql mysql 12582912 6月 10 18:38 ibtmp1
drwxr-x---. 2 mysql mysql 121 6月 10 18:14 lijin
drwxr-x---. 2 mysql mysql 4096 6月 10 18:38 mysql
-rw-r-----. 1 mysql mysql 321 6月 10 18:38 mysql-bin.000001
-rw-r-----. 1 mysql mysql 19 6月 10 18:38 mysql-bin.index
srwxrwxrwx. 1 mysql mysql 0 6月 10 18:38 mysql.sock
-rw-----. 1 mysql mysql 5 6月 10 18:38 mysql.sock.lock
drwxr-x---. 2 mysql mysql 8192 5月 13 18:17 performance_schema
-rw-----. 1 mysql mysql 1676 5月 13 18:17 private_key.pem
-rw-r--r--. 1 mysql mysql 452 5月 13 18:17 public_key.pem
-rw-r--r--. 1 mysql mysql 1112 5月 13 18:17 server-cert.pem
-rw-----. 1 mysql mysql 1680 5月 13 18:17 server-key.pem
drwxr-x---. 2 mysql mysql 8192 5月 13 18:17 sys
```